

Skip-gram Model in Word2Vec

Learning Distributed Word Representations from Local Context

Based on Mikolov et al. (2013) — Word2Vec Framework

A Comprehensive Research Paper with Mathematical Derivations

Abstract

The Skip-gram model, introduced as part of the Word2Vec framework by Mikolov et al. (2013), is a highly efficient method for learning high-quality distributed vector representations of words. By predicting surrounding context words given a target word, it captures rich semantic and syntactic relationships. This paper provides a clear, intuitive, and mathematically grounded explanation of the Skip-gram model, including its architecture, training process, negative sampling optimization, advantages, limitations, and comparison with the Continuous Bag-of-Words (CBOW) model. We also discuss its place in the evolution toward modern contextual embeddings such as ELMo and BERT.

1. Introduction

Word embeddings are dense, low-dimensional vector representations of words that encode semantic and syntactic information. They enable machines to understand that words like "king" and "queen" are semantically related, unlike traditional sparse representations that treat every word as an independent atomic symbol.

Traditional NLP methods suffered from the curse of dimensionality and failed to capture meaning. The Skip-gram model addresses these limitations by learning embeddings through a simple predictive task: given a center word, predict the words in its local context. This approach leverages the distributional hypothesis — words that appear in similar contexts tend to have similar meanings — to extract rich representations from raw unlabeled text.

This paper is organized as follows: Section 2 reviews limitations of traditional representations; Section 3 introduces the Skip-gram model; Sections 4–6 detail the mathematics, architecture, and training; Section 7 covers Negative Sampling; Sections 8–9 discuss advantages and limitations; Section 10 compares Skip-gram with CBOW; and Section 11 situates the model within the broader NLP landscape.

2. Background: Limitations of Traditional Representations

Before Skip-gram, the dominant word representations included:

- **One-hot Encoding:** Each word is a vector with a single 1 and all other entries 0. No semantic relationships are captured; "cat" and "dog" are as dissimilar as "cat" and "physics". Dimensionality equals vocabulary size V (often 50,000+).
 - **Bag-of-Words (BoW):** Represents documents by word counts, ignoring order and context. Loses all sequential and semantic information.
-

- **TF-IDF:** Weights terms by frequency and rarity across documents, but still produces sparse vectors without semantic similarity between words.

These methods treat words as atomic symbols and fail to model how meaning emerges from usage patterns. Skip-gram overcomes all three limitations by learning compact, dense vectors directly from context.

3. Skip-gram Model Overview

The core idea of Skip-gram is to predict context words from the target (center) word — the reverse of CBOW, which predicts the center from context. A sliding window of fixed size (typically 2–10 words) moves across the corpus. For each center word, all words within the window (excluding the center) become positive context examples.

Example — Sentence: "The quick brown fox jumps over the lazy dog." Center word: "fox" (window size = 2). Context words: "quick", "brown", "jumps", "over". Training pairs: (fox, quick), (fox, brown), (fox, jumps), (fox, over).

By optimizing these predictions across a large corpus, the model learns that semantically similar words have vectors that are close in the embedding space.



Figure 1. Sliding window mechanism for Skip-gram. The red box is the center word "fox"; blue boxes are context words within window size 2. Training pairs are shown below.

4. Mathematical Explanation

Let w_t be the center word and w_{t+j} a context word within the window of size c . The goal is to maximize the conditional probability $P(w_{t+j} | w_t)$.

4.1 Softmax Formulation

$$P(w_o | w_i) = \exp(v^T v^w_i) / \sum_{V=1} \exp(v^T v^w_i)$$

where v^w is the input embedding vector and v^w is the output embedding vector for word w , and V is the vocabulary size. The dot product $v^T v^w_i$ measures the similarity between the context word and the center word.

4.2 Objective Function

The objective function maximizes the average log probability over the entire corpus of T tokens:

$$J(\theta) = (1/T) \sum_{t=1}^T \sum_{-c \leq j \leq c, j \neq 0} \log P(w_{t+j} | w_t; \theta)$$

Training minimizes the negative log-likelihood $-J(\theta)$ via stochastic gradient descent (SGD) or Adam. The parameters θ are the input embedding matrix W ($V \times d$) and output matrix W' ($d \times V$).

5. Neural Network Architecture

Skip-gram uses a shallow, two-layer neural network. Despite its simplicity, this architecture learns remarkably powerful representations when trained on large corpora.

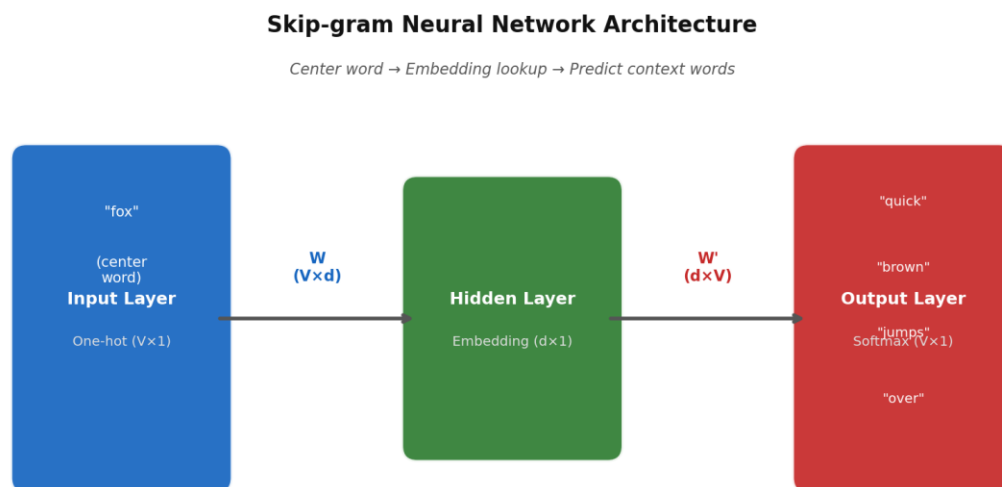


Figure 2. Skip-gram neural network architecture. The input layer receives the one-hot vector of the center word; the hidden layer is the embedding lookup (no activation function); the output layer applies softmax over the full vocabulary.

The three components are:

- **Input Layer:** One-hot vector of the center word (dimension V). Effectively an index lookup into the embedding matrix.
- **Hidden Layer (Embedding Layer):** d dimensions (typically 100–300). No non-linear activation — the hidden state is simply the row of W corresponding to the center word. This is the learned word vector.
- **Output Layer:** Softmax over all V vocabulary words, producing a probability distribution over context words.

The learned word embeddings are the rows of the input weight matrix W . Each of the d dimensions captures a latent semantic or syntactic feature. Notably, no non-linear activation is used — the power comes from the training objective, not architectural complexity.

6. Training Process

Training pairs are generated by sliding the window over the full corpus. For each (center, context) pair, the update procedure is:

1. Feed the one-hot center word through W to obtain its d -dimensional embedding.
2. Multiply the embedding by W' and apply softmax to produce a probability distribution over all V words.
3. Compute cross-entropy loss against the true context word (one-hot target).

- 4. Backpropagate the gradient and update W and W' via SGD.

This self-supervised process extracts massive supervision signal from unlabeled text — a corpus of 1 billion tokens with window size 5 generates up to 10 billion training pairs.

7. Optimization: Negative Sampling

Computing the full softmax requires normalizing over all V words at every update step — an $O(V)$ operation that is prohibitively slow for large vocabularies ($V \approx 10^5 - 10^6$). Negative Sampling (NS) approximates this by reformulating the task as binary classification.

For each positive pair (center, true context), k negative words are sampled from the unigram distribution raised to the $3/4$ power (which slightly boosts rare words). The NS objective is:

$$J_{NS} = -\log \sigma(v^T v^w) - \sum_{k=1} \log \sigma(-v^{neg_k T} v^w)$$

where $\sigma(x) = 1/(1+\exp(-x))$ is the sigmoid function. Only $(k+1)$ word vectors are updated per step instead of V , reducing complexity from $O(V)$ to $O(k)$. Typical values are $k = 5-20$ for small datasets and $k = 2-5$ for large corpora. Negative Sampling also often produces better embeddings than full softmax in practice.

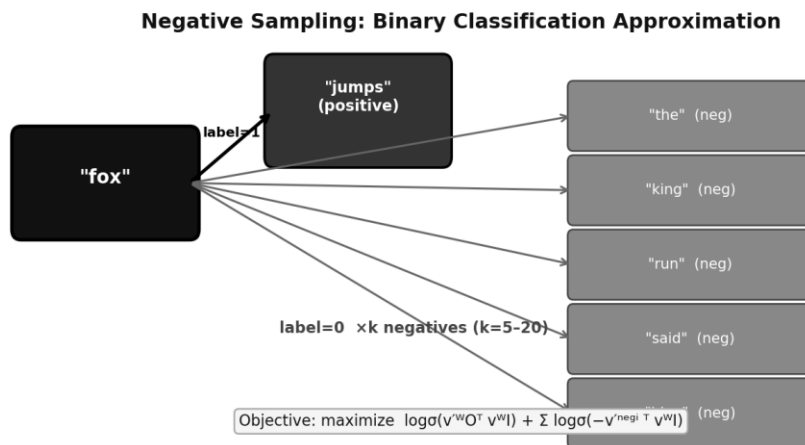


Figure 3. Negative Sampling: one positive pair (fox, jumps) and k negative samples are classified as 1 or 0. Only these $(k+1)$ vectors are updated per step, making training efficient.

8. Advantages of Skip-gram

- **Semantic arithmetic:** Vector offsets encode relationships: king – man + woman $\approx$ queen; Paris – France + Italy $\approx$ Rome.
- **Rare word performance:** Because Skip-gram generates many training pairs per center word, it learns high-quality embeddings even for infrequent terms — a significant advantage over CBOW.
- **Algorithm-agnostic:** Embeddings transfer easily across downstream tasks (classification, NER, MT) without task-specific retraining.
- **Scalability:** With Negative Sampling, training on billions of tokens is feasible on a single machine in hours.
- **No labels required:** Self-supervised training on raw text requires no manual annotation.

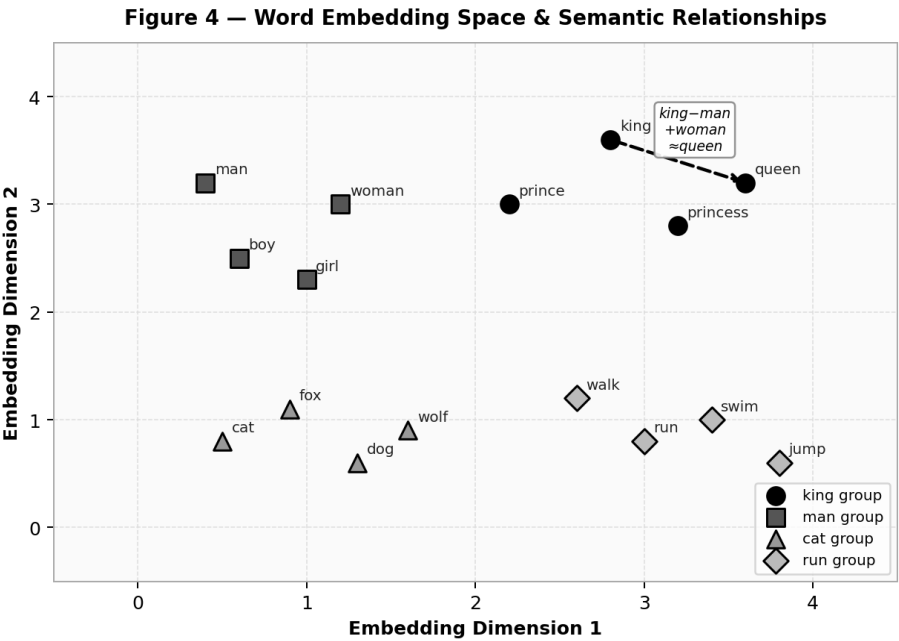


Figure 4. Word embedding space learned by Skip-gram. Semantically related words cluster together; vector arithmetic encodes analogies ($king - man + woman \approx queen$ shown by arrow).

9. Limitations

- **Static embeddings:** One fixed vector per word regardless of context. The word "bank" receives the same vector whether it means a financial institution or a river bank (polysemy problem).
- **Local context only:** The fixed window misses long-range dependencies that are crucial for understanding coreference, discourse, and complex syntax.
- **No subword information:** Out-of-vocabulary words and morphologically complex forms (e.g., "running" vs. "ran") receive no representation.
- **Corpus bias:** Embeddings inherit stereotypical associations present in the training corpus, which can reinforce societal biases.
- **Monolingual:** Standard Skip-gram trains separate embedding spaces per language; cross-lingual alignment requires additional techniques.

10. Comparison with CBOW

CBOW predicts the center word from the averaged embeddings of its context words, while Skip-gram predicts each context word from the center. The two models have complementary strengths:

Table 1. Skip-gram vs CBOW comparison.

Property	Skip-gram	CBOW
Task	Center → Context	Context → Center
Rare words	Superior	Weaker
Frequent words	Adequate	Slightly better

Property	Skip-gram	CBOW
Training speed	Slower	Faster (~3×)
Semantic analogy	Excellent	Good
Typical use case	General NLP tasks	Large frequent-word corpora

In practice, Skip-gram typically produces higher-quality embeddings for downstream tasks, especially when the vocabulary contains many rare or domain-specific terms.

Figure 5 — Skip-gram vs CBOW Architecture & Model Evolution

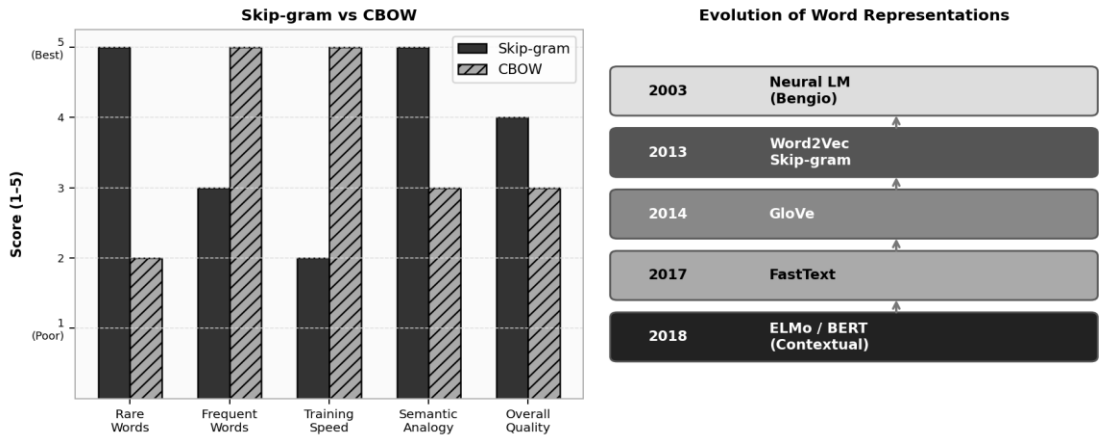


Figure 5. Left: qualitative comparison of Skip-gram vs CBOW across five criteria (scores 1–5). Right: timeline of word representation methods from Neural LM (2003) through contextual models (2018).

11. Modern Context and Successors

Skip-gram inspired a generation of improved representations:

- **GloVe (Pennington et al., 2014):** Combines local Skip-gram-style context with global co-occurrence matrix factorization, achieving strong performance on analogy benchmarks.
- **FastText (Bojanowski et al., 2017):** Extends Skip-gram with character n-gram subword embeddings, enabling representation of morphologically rich languages and out-of-vocabulary words.
- **ELMo (Peters et al., 2018):** Contextual embeddings from bidirectional LSTMs trained on language modeling. Each token receives a different vector depending on sentence context, solving polysemy.
- **BERT (Devlin et al., 2019):** Transformer-based contextual embeddings pre-trained with masked language modeling. Dominates nearly all NLP benchmarks and is the direct descendant of Skip-gram's self-supervised philosophy.

While transformers now dominate the NLP landscape, Skip-gram remains foundational. Its core insight — that predicting context from a self-supervised signal yields semantically rich representations — directly inspired BERT's masked language model pre-training objective.

12. Conclusion

The Skip-gram model demonstrated that simple predictive tasks on raw text can yield powerful distributed representations. Its elegant design, efficient Negative Sampling optimization, and strong semantic capture made it a cornerstone of modern NLP. Despite limitations such as static vectors and local-window context, its principles continue to influence today's large language models.

Understanding Skip-gram provides essential intuition for representation learning, self-supervision, and the power of context in language — concepts that remain central to the largest models of 2025 and beyond.

References

- Bengio, Y., Ducharme, R., Vincent, P., & Janvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, 1137–1155. <https://www.jmlr.org/papers/v3/bengio03a.html>
- Bojanowski, P., Grave, E., Joulin, A., & Mikolov, T. (2017). Enriching word vectors with subword information. *Transactions of the Association for Computational Linguistics*, 5, 135–146. https://doi.org/10.1162/tacl_a_00051
- Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of deep bidirectional transformers for language understanding. *Proceedings of NAACL-HLT 2019*, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>
- Mikolov, T., Chen, K., Corrado, G., & Dean, J. (2013). Efficient estimation of word representations in vector space. *arXiv preprint arXiv:1301.3781*. <https://arxiv.org/abs/1301.3781>
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. S., & Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in Neural Information Processing Systems*, 26, 3111–3119. <https://proceedings.neurips.cc/paper/2013/hash/9aa42b31882ec039965f3c4923ce901b-Abstract.html>
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of EMNLP 2014*, 1532–1543. <https://doi.org/10.3115/v1/D14-1162>
- Peters, M. E., Neumann, M., Iyyer, M., Gardner, M., Clark, C., Lee, K., & Zettlemoyer, L. (2018). Deep contextualized word representations. *Proceedings of NAACL-HLT 2018*, 2227–2237. <https://doi.org/10.18653/v1/N18-1202>
- Rong, X. (2014). word2vec parameter learning explained. *arXiv preprint arXiv:1411.2738*. <https://arxiv.org/abs/1411.2738>